DOCUMENTACIÓN DE INGENIERÍA DE SOFTWARE

MANUAL OFICIAL DE ARQUITECTURA & CÓDIGO

MANUAL DEL DESARROLLADOR

Arquitectura del Stack Laravel 12 + Tailwind v4 + Alpine.js, Topología de Servicios, Mecanismos de Cifrado, Integraciones de Terceros (Wompi, Envía, Factus), Servidor MCP y Pipeline de Pruebas.

VERSIÓN DEL SISTEMA
FusaShop v2.4-LTS (Edición Enterprise)

FRAMEWORK & RUNTIME
Laravel 12.x • PHP 8.2+ • Tailwind v4

FECHA DE COMPILACIÓN
Octubre 2026

ENTORNO CERTIFICADO
76 / 76 Tests Pasados (Pest / PHPUnit)

© 2026 FusaShop — Comercio Electrónico Sostenible de Fusagasugá

Confidencial • Uso Exclusivo del Equipo de Tecnología

Contenido del Documento

Este manual describe la arquitectura interna, decisiones técnicas, convenciones de ingeniería y lineamientos operacionales requeridos para extender, mantener y desplegar la plataforma **FusaShop**.

1. Ecosistema Tecnológico y Decisiones de Arquitectura — Capítulo 1
2. Topología del Proyecto y Estructura de Directorios — Capítulo 2
3. Modelado de Datos y Capa de Persistencia Eloquent — Capítulo 3
4. Capa de Servicios del Dominio & Lógica de Negocio — Capítulo 4
5. Integraciones de Terceros: Wompi, Envía y Factus DIAN — Capítulo 5
6. Asistente Inteligente FusaBot y Servidor MCP (JSON-RPC) — Capítulo 6
7. Seguridad, Cifrado Criptográfico y Control de Acceso — Capítulo 7
8. Guía de Instalación Local, Configuración y Workflows — Capítulo 8
9. Estrategia de Testing Automatizado y Aseguramiento de Calidad — Capítulo 9
10. Estándares de Codificación, Versionado y Convenciones Git — Capítulo 10

> 💡 **Principio Rector del Código en FusaShop:** "Mantenemos una arquitectura limpia orientada a servicios, donde los controladores actúan estrictamente como coordinadores de peticiones HTTP, delegando las transacciones complejas a clases de servicio especializadas (`app/Services`), los procesos diferidos a colas asíncronas (`app/Jobs`), y la persistencia a modelos Eloquent con tipado estricto."

1. Ecosistema Tecnológico y Decisiones de Arquitectura

FusaShop está construido sobre un stack moderno y eficiente diseñado para ofrecer alta disponibilidad, respuesta ultrarrápida en dispositivos móviles de gama baja y media, y facilidad de mantenimiento sin sobreingeniería.

COMPONENTE	TECNOLOGÍA SELECCIONADA	VERSIÓN	PROPÓSITO EN FUSASHOP
Backend Core	PHP / Laravel Framework	PHP 8.2+ / Laravel 12.x	Enrutamiento, Middleware de seguridad, Eloquent ORM, Jobs y Sistema de Eventos.
Frontend Estilos	Tailwind CSS	v4.0 (Engine JIT)	Diseño modular responsivo con paleta corporativa nativa (Emerald, Mint, Orange).
Frontend Reactividad	Alpine.js	v3.13+	Interactividad ligera en el cliente (modales, carritos flotantes, menús drawer, calculadoras).
Base de Datos	SQLite (Dev/CI) / MySQL / PostgreSQL	MySQL 8.0+ / PgSQL 15+	Almacenamiento relacional con integridad referencial, índices espaciales y transacciones ACID.
Caché & Sesiones	Redis / Database / File	Redis 7.x / DB Fallback	Gestión de tokens OAuth2, throttling de peticiones y persistencia de carritos.
Optimización de Medios	WebP Engine / cwebp	Formatos WebP / PNG / JPG	Compresión del 65%-80% en imágenes con fallback automático transparente.

1.1 ¿Por qué Laravel 12 + Alpine.js en lugar de una SPA pesada?

Muchos comercios de Fusagasugá y clientes finales operan en redes móviles 3G/4G con planes de datos limitados. Las Single Page Applications basadas en pesados bundles de React o Angular transfieren megabytes de JavaScript en la carga inicial, lo que genera retrasos inaceptables (First Contentful Paint > 4s). FusaShop adopta la estrategia **SSR + Micro-interacciones reactivas**:

🚀 **Server-Side Rendering (Blade)**

HTML puro generado en el servidor en menos de 65 ms. Indexable de forma instantánea por motores de búsqueda (SEO local) con metadatos OpenGraph dinámicos.

⚡ **Alpine.js Declarativo**

Un único script de ~15 KB maneja toda la reactividad del cliente (filtros de catálogo, apertura de drawers, cálculo dinámico de FusaCoins y validaciones de formulario).

2. Topología del Proyecto y Organización de Directorios

La estructura del código sigue estrictamente las mejores prácticas de Laravel 12, desacoplando los dominios de negocio en namespaces independientes:

ESTRUCTURA DE DIRECTORIOS CLAVE FUSASHOP/

```
fusashop/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Analyst/         # Controladores del Administrador / Analista de Datos
│   │   │   │   ├── AnalystController.php
│   │   │   │   └── IntegrationController.php
│   │   │   ├── Merchant/        # Gestión de Tiendas, Inventario, Envíos y Finanzas
│   │   │   │   ├── MerchantController.php
│   │   │   │   └── CouponController.php
│   │   │   ├── Consumer/        # Catálogo, Carrito, Checkout, Direcciones y FusaBot
│   │   │   │   ├── ConsumerController.php
│   │   │   │   ├── CheckoutController.php
│   │   │   │   └── AddressController.php
│   │   │   ├── Webhooks/        # Endpoints de Recepción Asíncrona (Wompi, Envía)
│   │   │   ├── McpController.php   # Servidor Model Context Protocol (JSON-RPC)
│   │   │   └── ChatController.php  # Mensajería interna y soporte
│   │   └── Middleware/          # SecurityHeaders, EnsureKycApproved, CompressResponse
│   ├── Jobs/                    # Tareas en segundo plano (Envía, Factus, Wompi)
│   ├── Models/                  # Entidades Eloquent (User, Order, Product, etc.)
│   ├── Services/                # Lógica pura del dominio
│   │   ├── Admin/               # Servicios de Inteligencia Territorial y Diagnósticos
│   │   ├── CheckoutService.php
│   │   ├── EnviaService.php
│   │   ├── FactusService.php
│   │   ├── OrderPaymentService.php
│   │   ├── StockService.php
│   │   └── WompiService.php
├── database/
│   ├── migrations/              # 37 migraciones versionadas
│   └── seeders/                 # Datos de prueba de tiendas locales y productos
├── resources/
│   ├── css/app.css              # Estilos Tailwind v4 y directivas responsivas
│   └── views/                    # Plantillas Blade segmentadas por rol
└── tests/Feature/               # 76 pruebas de integración y aceptación
```

2.1 Convenciones de Código y Nomenclatura

- **Controladores:** Nombre singular + sufijo `Controller` (ej. `CheckoutController`). Deben mantenerse ligeros (< 200 líneas preferentemente).
- **Modelos:** Nombre singular en inglés (ej. `Product`, `BannerRequest`, `CompanyProfile`). Todas las tablas asociadas utilizan plural snake_case.

- **Servicios:** Clases independientes en `AppServices` con inyección de dependencias en constructores o métodos de ejecución directa.
- **Migraciones:** Estrictamente idempotentes con definición precisa de tipos de datos, llaves foráneas y borrados en cascada condicionados.

3. Modelado de Datos y Capa de Persistencia

FusaShop implementa un esquema relacional optimizado para e-commerce multitienda georreferenciado. A continuación se presentan las entidades primordiales y sus relaciones estructurales:

MODELO	TABLA ASOCIADA	RELACIONES CLAVE	CAMPOS CRÍTICOS & ATRIBUTOS

`User`	`users`	`hasOne(CompanyProfile)`
		`hasMany(Order)`
		`hasMany(Review)`
			`role` ('consumer','merchant','analyst'), `loyalty_points` (FusaCoins), `email_verified_code`, `is_blocked`.

`CompanyProfile`	`company_profiles`	`belongsTo(User, 'merchant_id')`	`store_name`, `kyc_status`, `wompi_public_key`, `wompi_private_key` (cifrado), `latitude`, `longitude`.
`Product`	`products`	`belongsTo(User, 'merchant_id')` `hasMany(OrderItem)`	`price`, `discount_price`, `stock`, `weight`, `specifications` (JSON), `allows_points` (booleano).
`Order`	`orders`	`belongsTo(User, 'consumer_id')` `hasMany(OrderItem)`	`reference` ('FS-...'), `status`, `payment_status`, `total`, `points_discount`, `shipping_tracking_number`.
`OrderItem`	`order_items`	`belongsTo(Order)` `belongsTo(Product)`	`product_id`, `quantity`, `price`, `subtotal`.
`BannerRequest`	`banner_requests`	`belongsTo(User, 'merchant_id')`	`title`, `status` ('pending','approved','rejected'), `target_sector`, `cost_points`, `starts_at`, `expires_at`.
`PQRS`	`p_q_r_s`	`belongsTo(User)` `belongsTo(Order)`	`ticket_number`, `type` ('peticion','queja','reclamo','sugerencia'), `status` ('open','in_progress','closed').

3.1 Cifrado Criptográfico Transparente en Modelos

Para proteger la información financiera de los comerciantes en caso de una fuga en la base de datos, los secretos de pasarela se cifran con **AES-256-CBC** utilizando los casts nativos de Laravel en el modelo `CompanyProfile`:

APP/MODELS/COMPANYPROFILE.PHP — CIFRADO DE CREDENCIALES

```php
protected $casts = [
    'wompi_private_key'     => 'encrypted',
    'wompi_integrity_secret'=> 'encrypted',
    'is_verified'           => 'boolean',
    'kyc_approved_at'       => 'datetime',
    'latitude'              => 'float',
```

```
    'longitude'             => 'float',
];
```

4. Capa de Servicios del Dominio & Lógica de Negocio

FusaShop centraliza las transacciones críticas en clases de servicio puras. Esto garantiza que las pruebas unitarias y de integración puedan ejecutarse de forma aislada sin simular controladores HTTP.

4.1 Gestión de Stock Atómica (`StockService`)

Evita condiciones de carrera (*race conditions*) y sobreventas en momentos de alto tráfico mediante transacciones con bloqueo pesimista en base de datos:

APP/SERVICES/STOCKSERVICE.PHP — RESERVA SEGURA DE INVENTARIO

```php
public function reserveStockForOrder(Order $order): void
{
    DB::transaction(function () use ($order) {
        foreach ($order->items as $item) {
            // Bloqueo exclusivo de fila hasta finalizar la transacción
            $product = Product::where('id', $item->product_id)->lockForUpdate()->firstOrFail();

            if ($product->stock < $item->quantity) {
                throw new RuntimeException("Stock insuficiente para: {$product->name}");
            }

            $product->decrement('stock', $item->quantity);

            // Disparo de alerta si el stock baja del umbral mínimo
            if ($product->stock <= $product->low_stock_threshold) {
                SendLowStockAlert::dispatch($product);
            }
        }
    });
}
```

4.2 Servicios de Analítica Avanzada (`app/Services/Admin/`)

El portal administrativo cuenta con motores de inteligencia de negocios para diagnóstico del comercio municipal:

- `ApiHealthService`: Sondea la disponibilidad de Wompi, Envía, Factus y el driver de correo con caché de 60 segundos.
- `DashboardMetricsService`: Calcula Gross Merchandise Value (GMV), ticket promedio, retención de clientes y comerciantes activos.
- `SectorCensusService`: Agrupa negocios por comunas y sectores de Fusagasugá para reportes de desarrollo económico local.
- `TerritoryHeatmapService`: Procesa coordenadas GPS de órdenes entregadas para generar mapas de calor logísticos.

5. Integraciones de Terceros: Wompi, Envía y Factus

5.1 Wompi (Pasarela de Pagos Multitienda)

FusaShop admite pagos directos a las cuentas Wompi individuales de cada comerciante. Para garantizar la inviolabilidad del monto cobrado, se genera una firma criptográfica SHA-256:

APP/SERVICES/WOMPISERVICE.PHP — FIRMA DE INTEGRIDAD CRIPTOGRÁFICA

```php
/**
 * Firma de integridad: SHA256("{referencia}{monto_en_centavos}{moneda}{secreto_integridad}")
 */
public function integritySignature(string $reference, int $amountInCents, string $currency = 'COP', ?string $cust
{
    $secret = $customSecret;
    if ($secret === null) {
        $order = Order::where('reference', $reference)->first();
        $secret = $order?->merchantProfile()?->wompi_integrity_secret ?: config('services.wompi.integrity_secret'
    }

    return hash('sha256', $reference . $amountInCents . $currency . $secret);
}
```

5.2 Envía Colvanes (Logística Nacional y Guías)

El servicio `EnviaService` implementa cálculo de volumen cúbico y consolidación automática de ítems. Una vez el pago es confirmado por el webhook de Wompi, el sistema encola automáticamente el Job `GenerateEnviaShipment` para generar la guía en PDF sin bloquear al cliente.

5.3 Factus (Facturación Electrónica DIAN)

Emisión conforme a los requisitos fiscales de la DIAN en Colombia. Implementa token bearer OAuth2 con caché inteligente:

APP/SERVICES/FACTUSSERVICE.PHP — AUTENTICACIÓN OAUTH2 CON CACHÉ

```php
private function token(bool $forceRefresh = false): string
{
    if ($forceRefresh) {
        Cache::forget('factus.access_token');
    }

    return Cache::remember('factus.access_token', 50 * 60, function () {
        $cfg = config('services.factus');
        $res = Http::baseUrl($cfg['api_url'])->asForm()->acceptJson()->post('/oauth/token', [
            'grant_type'    => 'password',
            'client_id'     => $cfg['client_id'],
            'client_secret' => $cfg['client_secret'],
```

```
            'username'      => $cfg['username'],
            'password'      => $cfg['password'],
        ]);
        return $res->json('access_token');
    });
}
```

6. Asistente Inteligente FusaBot y Servidor MCP

FusaShop integra dos mecanismos de Inteligencia Artificial: un asistente contextual embebido para el usuario web (**FusaBot**) y un servidor oficial **Model Context Protocol (MCP)** sobre protocolo JSON-RPC 2.0 para agentes autónomos externos.

6.1 Triaje Contextual de FusaBot (`ConsumerController::assistantSearch`)

El bot analiza la intención de la consulta y el rol del usuario para proporcionar respuestas de resolución directa:

- **Rol Comerciante:** Guía interactiva sobre resolución de errores en emisión de CUFE DIAN, configuración de llaves Wompi, y solicitudes de banners publicitarios.
- **Rol Administrador:** Resumen instantáneo de ventas globales del día, pedidos en estado pendiente, trámites KYC por auditar y tickets PQRS abiertos.
- **Rol Comprador:** Diagnóstico de errores de pago (PSE, Wompi, tarjetas), rastreo en tiempo real de paquetes y motor semántico de búsqueda con renderizado directo de imágenes de productos.

6.2 Servidor Model Context Protocol (MCP)

FusaShop expone herramientas estandarizadas mediante `POST /api/mcp` autenticado mediante token Bearer (`MCP_TOKEN`):

HERRAMIENTA MCP	PARÁMETROS DE ENTRADA	DESCRIPCIÓN Y RETORNO
`get_order`	`reference: "FS-..."`	Retorna el estado de pago, tracking de envío, ítems, totales y número de factura DIAN.
`search_catalog`	`query, category, limit`	Búsqueda en catálogo con stock disponible, precios vigentes y slugs para enlace directo.
`validate_shipping`	`street, city, state, postal_code`	Valida la nomenclatura de direcciones colombianas y cotiza el flete logístico vía Envía.

EJEMPLO DE LLAMADA JSON-RPC 2.0 AL SERVIDOR MCP

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_order",
    "arguments": { "reference": "FS-20261019-9482" }
  }
}
```

7. Seguridad, Cifrado y Control de Acceso

7.1 Pipeline de Middlewares de Seguridad

Cada petición HTTP entrante atraviesa una estricta cadena de validación configurada en `bootstrap/app.php`:

🛡️ SecurityHeaders

Inyecta cabeceras HTTP de endurecimiento: `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: strict-origin-when-cross-origin` y políticas CSP.

👥 RoleMiddleware

Protege rutas administrativas y comerciales verificando `Auth::user()->role === $role` con redirección automática si no cuenta con privilegios.

📜 EnsureKycApproved

Impide que comercios no verificados puedan publicar productos o recibir pagos en línea antes de la validación legal de su RUT/Cédula.

🗜️ CompressResponse

Aplica compresión Gzip transparente en payloads JSON y respuestas HTML cuando el cliente lo soporta (`Accept-Encoding: gzip`).

7.2 Verificación de 6 Dígitos por Correo (OTP)

Al registrarse un nuevo usuario, el sistema genera un código numérico aleatorio de 6 dígitos con expiración de 15 minutos almacenado en la tabla `users` (`verification_code` y `verification_expires_at`). El usuario no puede realizar transacciones hasta validar dicho código.

7.3 Registro Inmutable de Auditoría (`AdminAuditLog`)

Cualquier acción crítica ejecutada por un Administrador (aprobación/rechazo de KYC, bloqueo de tiendas, reversión de pagos) se registra de forma inmutable con la IP de origen, User Agent y snapshot del estado anterior y posterior en formato JSON.

8. Guía de Instalación Local y Flujo de Trabajo

Para levantar el entorno completo de desarrollo en una estación de trabajo Windows, Linux o macOS, ejecute la siguiente secuencia estandarizada:

8.1 Prerrequisitos de la Estación

- **PHP 8.2 o superior** con extensiones: `pdo`, `pdo_sqlite`, `pdo_mysql`, `curl`, `mbstring`, `openssl`, `fileinfo`, `bcmath`.
- **Composer 2.6+**
- **Node.js 18+** y **npm 9+**
- **Git**

8.2 Secuencia de Comandos

COMANDOS DE INSTALACIÓN PASO A PASO

```
# 1. Clonar el repositorio
git clone https://github.com/fusashop/fusashop.git
cd fusashop

# 2. Instalar dependencias PHP
composer install

# 3. Instalar dependencias frontend de Node
npm install

# 4. Configurar variables de entorno
cp .env.example .env
php artisan key:generate

# 5. Ejecutar migraciones y poblado de datos iniciales
php artisan migrate --seed

# 6. Crear enlace simbólico de almacenamiento público
php artisan storage:link

# 7. Compilar activos CSS con Tailwind v4
npm run build

# 8. Iniciar el servidor local de desarrollo
php artisan serve
```

8.3 Optimización de Recursos Gráficos a WebP

El proyecto incluye un script automatizado para procesar y optimizar todos los banners e imágenes del sistema a formato moderno WebP:

COMPRESIÓN DE IMÁGENES WEBP

```
node scripts/optimize-images.cjs
```

9. Estrategia de Testing Automatizado (76/76 Tests)

FusaShop cuenta con una robusta suite de pruebas unitarias y de integración en `tests/Feature/` que validan de forma integral el correcto funcionamiento de la plataforma:

SUITE DE PRUEBAS	ARCHIVO DE TEST	COBERTURA & VALIDACIONES
Seguridad & Gobernanza	`AdminSecurityAndGovernanceTest.php`	Auditoría de logs, bloqueo de tiendas fraudulentas, autorización RBAC.
Checkout & FusaCoins	`SelectiveCheckoutAndFusaCoinsTest.php`	Cálculo de descuentos de puntos, validación de stock y cobros Wompi.
Verificación 6 Dígitos	`SixDigitEmailVerificationAndNotificationsTest.php`	Envío de OTP, expiración temporal y restricción de compras sin verificación.
Publicidad y Banners	`MerchantBannerAdvertisingTest.php`	Solicitudes de publicidad, aprobación administrativa y geolocalización.
Asistente FusaBot	`FusaBotAssistantTest.php`	Triaje semántico de respuestas, recomendaciones con foto y escalamiento.
Finanzas de Tiendas	`MerchantSalesReportCenterTest.php`	Reportes de ventas, comisiones, pagos netos y exportación de balance.

EJECUCIÓN DEL TEST SUITE COMPLETO

```
# Ejecutar la suite completa garantizando 0 fallos
php artisan test

# Salida esperada:
PASS  TestsFeatureAdminSecurityAndGovernanceTest
PASS  TestsFeatureCheckoutFlowTest
PASS  TestsFeatureFusaBotAssistantTest
...
Tests:    76 passed (182 assertions)
Duration: 8.42s
```

10. Estándares Git Flow y Contribución

- **Ramas Principales:** `main` (código de producción estrictamente estable y firmado) y `develop` (integración continua de nuevas características).
- **Ramas de Trabajo:** `feature/nombre-modulo`, `fix/nombre-bug`, `perf/optimizacion`.

- **Mensajes de Commit Convencionales:** `feat: añade integración con Envía`, `fix: corrige cálculo de IVA en Factus`, `perf: optimiza compresión WebP`.
- **Pull Requests:** Todo merge requiere que el pipeline de GitHub Actions / GitLab CI pase el 100% de los 76 tests y la validación de estilo PSR-12.